MANAV RACHNA INSTITUTE OF INTERNATIONAL RESEARCH AND STUDIES

NAME - MEENAL

CLASS - MCA 1 A

ROLL NO - 26MCAP101N042

SUBMITTED BY - MEENAL

SUBMITTED TO - DR SACHIN SHARMA

Program 1

Write a program to perform various operations in the list: Insertion, Deletion, Display

Code:

```
numbers = []

while True:
    print("\n1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value to insert: "))
        pos = int(input("Enter position (0 to " + str(len(numbers)) + "): "))
        if pos < 0 or pos > len(numbers):
            print("Invalid position")
        else:
            numbers.insert(pos, value)
            print("Value inserted")
    elif choice == 2:
        if len(numbers) == 0:
            print("List is empty")
        else:
            value = int(input("Enter value to delete: "))
            if value in numbers:
                numbers.remove(value)
                print("Value deleted")
            else:
                print("Value not found")
    elif choice == 3:
        if len(numbers) == 0:
            print("List is empty")
        else:
            print("List:", numbers)
    elif choice == 4:
        print("Program ended")
        break
    else:
        print("Invalid choice")
```

Output:

```
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 10
Enter position (0 to 0): 0
Value inserted

1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 20
Enter position (0 to 1): 1
Value inserted

1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 30
Enter position (0 to 2): 2
Value inserted

1. Insert
```

```
2. Delete
3. Display
4. Exit
Enter your choice: 3
List: [10, 20, 30]

1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 2
Enter value to delete: 20
Value deleted

1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 2
Enter value to delete: 99
Value not found

1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 5
Enter position (0 to 2): 7
Invalid position

1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
List: [10, 30]

1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 4
Program ended
```

Program 2

Write a program to sort the list using Quick Sort and Merge Sort. Also, find the comparison on the basis of time complexity

Code:

```
import time
import random

q_comp = 0
m_comp = 0


def quick_sort(arr):
    global q_comp
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = []
    middle = []
    right = []
    for x in arr:
        q_comp += 1
        if x < pivot:
            left.append(x)
        elif x == pivot:
            middle.append(x)
        else:
            right.append(x)
    return quick_sort(left) + middle + quick_sort(right)


def merge_sort(arr):
    global m_comp
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        m_comp += 1
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result = result + left[i:]
    result = result + right[j:]
    return result


n = int(input("Enter number of elements: "))
data = []
for i in range(n):
    data.append(int(input("Enter element " + str(i + 1) + ": ")))

print("\nOriginal list:", data)
print("Quick sort result:", quick_sort(data[:]))
print("Merge sort result:", merge_sort(data[:]))

size = int(input("\nEnter size of random list for timing: "))
big = [random.randint(1, 100000) for i in range(size)]

q_comp = 0
start = time.perf_counter()
quick_sort(big[:])
q_time = time.perf_counter() - start
m_comp = 0
```

```
start = time.perf_counter()
merge_sort(big[:])
m_time = time.perf_counter() - start

print("\nComparison for", size, "elements")
print("%-12s %-15s %-12s" % ("Algorithm", "Time (sec)", "Comparisons"))
print("%-12s %-15.6f %-12d" % ("Quick Sort", q_time, q_comp))
print("%-12s %-15.6f %-12d" % ("Merge Sort", m_time, m_comp))

print("\nTime complexity")
print("%-12s %-12s %-12s %-12s" % ("Algorithm", "Best", "Average", "Worst"))
print("%-12s %-12s %-12s %-12s" % ("Quick Sort", "O(n log n)", "O(n log n)", "O(n^2)"))
print("%-12s %-12s %-12s %-12s" % ("Merge Sort", "O(n log n)", "O(n log n)", "O(n log n)"))
```

Output:

```
Enter number of elements: 6
Enter element 1: 45
Enter element 2: 12
Enter element 3: 78
Enter element 4: 3
Enter element 5: 12
Enter element 6: 56

Original list: [45, 12, 78, 3, 12, 56]
Quick sort result: [3, 12, 12, 45, 56, 78]
Merge sort result: [3, 12, 12, 45, 56, 78]

Enter size of random list for timing: 2000

Comparison for 2000 elements
Algorithm    Time (sec)      Comparisons
Quick Sort   0.003583        27827
Merge Sort   0.005292        19416

Time complexity
Algorithm    Best         Average      Worst
Quick Sort   O(n log n)   O(n log n)   O(n^2)
Merge Sort   O(n log n)   O(n log n)   O(n log n)
```

Program 3

Write a program to perform all operations for the Stack using an Array, the Queue using an Array and the Circular Queue using an Array. Implement browser history using stacks

Code:

```
size = 5


def stack_menu():
    stack = [None] * size
    top = -1
    while True:
        print("\n--- Stack using Array ---")
        print("1. Push  2. Pop  3. Peek  4. Display  5. Back")
        ch = int(input("Enter choice: "))
        if ch == 1:
            if top == size - 1:
                print("Stack Overflow")
            else:
                top += 1
                stack[top] = int(input("Enter value: "))
        elif ch == 2:
            if top == -1:
                print("Stack Underflow")
            else:
                print("Popped:", stack[top])
                top -= 1
        elif ch == 3:
            if top == -1:
                print("Stack is empty")
            else:
                print("Top element:", stack[top])
        elif ch == 4:
            if top == -1:
                print("Stack is empty")
            else:
                print("Stack (top to bottom):", end=" ")
                for i in range(top, -1, -1):
                    print(stack[i], end=" ")
                print()
        elif ch == 5:
            break
        else:
            print("Invalid choice")


def queue_menu():
    queue = [None] * size
    front = 0
    rear = -1
    while True:
        print("\n--- Queue using Array ---")
        print("1. Enqueue  2. Dequeue  3. Display  4. Back")
        ch = int(input("Enter choice: "))
        if ch == 1:
            if rear == size - 1:
                print("Queue is full")
            else:
                rear += 1
                queue[rear] = int(input("Enter value: "))
        elif ch == 2:
            if front > rear:
                print("Queue is empty")
            else:
                print("Dequeued:", queue[front])
                front += 1
        elif ch == 3:
            if front > rear:
                print("Queue is empty")
            else:
                print("Queue:", end=" ")
```

```
                for i in range(front, rear + 1):
                    print(queue[i], end=" ")
                print()
        elif ch == 4:
            break
        else:
            print("Invalid choice")


def circular_menu():
    cq = [None] * size
    front = 0
    rear = -1
    count = 0
    while True:
        print("\n--- Circular Queue using Array ---")
        print("1. Enqueue  2. Dequeue  3. Display  4. Back")
        ch = int(input("Enter choice: "))
        if ch == 1:
            if count == size:
                print("Circular queue is full")
            else:
                rear = (rear + 1) % size
                cq[rear] = int(input("Enter value: "))
                count += 1
        elif ch == 2:
            if count == 0:
                print("Circular queue is empty")
            else:
                print("Dequeued:", cq[front])
                front = (front + 1) % size
                count -= 1
        elif ch == 3:
            if count == 0:
                print("Circular queue is empty")
            else:
                print("Circular queue:", end=" ")
                i = front
                for k in range(count):
                    print(cq[i], end=" ")
                    i = (i + 1) % size
                print()
        elif ch == 4:
            break
        else:
            print("Invalid choice")


def browser_menu():
    back_stack = []
    forward_stack = []
    current = "Home"
    while True:
        print("\n--- Browser History ---")
        print("Current page:", current)
        print("1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu")
        ch = int(input("Enter choice: "))
        if ch == 1:
            page = input("Enter page name: ")
            back_stack.append(current)
            current = page
            forward_stack = []
        elif ch == 2:
            if len(back_stack) == 0:
                print("No previous page")
            else:
                forward_stack.append(current)
                current = back_stack.pop()
        elif ch == 3:
            if len(forward_stack) == 0:
                print("No forward page")
            else:
                back_stack.append(current)
```

```
                current = forward_stack.pop()
        elif ch == 4:
            print("Back history:", back_stack)
            print("Current:", current)
            print("Forward history:", forward_stack[::-1])
        elif ch == 5:
            break
        else:
            print("Invalid choice")


while True:
    print("\n===== MAIN MENU =====")
    print("1. Stack")
    print("2. Queue")
    print("3. Circular Queue")
    print("4. Browser History")
    print("5. Exit")
    option = int(input("Enter your option: "))
    if option == 1:
        stack_menu()
    elif option == 2:
        queue_menu()
    elif option == 3:
        circular_menu()
    elif option == 4:
        browser_menu()
    elif option == 5:
        print("Program ended")
        break
    else:
        print("Invalid option")
```

Output:

```
===== MAIN MENU =====
1. Stack
2. Queue
3. Circular Queue
4. Browser History
5. Exit
Enter your option: 1

--- Stack using Array ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 1
Enter value: 10

--- Stack using Array ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 1
Enter value: 20

--- Stack using Array ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 1
Enter value: 30

--- Stack using Array ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 4
Stack (top to bottom): 30 20 10

--- Stack using Array ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 3
Top element: 30

--- Stack using Array ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 2
Popped: 30

--- Stack using Array ---
```

```
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 4
Stack (top to bottom): 20 10

--- Stack using Array ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 2
Popped: 20

--- Stack using Array ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 2
Popped: 10

--- Stack using Array ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 2
Stack Underflow

--- Stack using Array ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 5

===== MAIN MENU =====
1. Stack
2. Queue
3. Circular Queue
4. Browser History
5. Exit
Enter your option: 2

--- Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 11

--- Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 22

--- Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 33

--- Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 3
Queue: 11 22 33

--- Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 2
Dequeued: 11

--- Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 2
Dequeued: 22

--- Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 3
Queue: 33

--- Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 4

===== MAIN MENU =====
1. Stack
2. Queue
```

```
3. Circular Queue
4. Browser History
5. Exit
Enter your option: 3

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 1

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 2

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 3

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 4

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 5

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Circular queue is full

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 3
Circular queue: 1 2 3 4 5

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 2
Dequeued: 1

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 2
Dequeued: 2

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 7

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 3
Circular queue: 3 4 5 7

--- Circular Queue using Array ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 4

===== MAIN MENU =====
1. Stack
2. Queue
3. Circular Queue
4. Browser History
5. Exit
Enter your option: 4

--- Browser History ---
```

```
Current page: Home
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 1
Enter page name: google.com

--- Browser History ---
Current page: google.com
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 1
Enter page name: youtube.com

--- Browser History ---
Current page: youtube.com
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 1
Enter page name: github.com

--- Browser History ---
Current page: github.com
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 4
Back history: ['Home', 'google.com', 'youtube.com']
Current: github.com
Forward history: []

--- Browser History ---
Current page: github.com
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 2

--- Browser History ---
Current page: youtube.com
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 2

--- Browser History ---
Current page: google.com
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 4
Back history: ['Home']
Current: google.com
Forward history: ['youtube.com', 'github.com']

--- Browser History ---
Current page: google.com
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 3

--- Browser History ---
Current page: youtube.com
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 1
Enter page name: wikipedia.org

--- Browser History ---
Current page: wikipedia.org
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 4
Back history: ['Home', 'google.com', 'youtube.com']
Current: wikipedia.org
Forward history: []

--- Browser History ---
Current page: wikipedia.org
1. Visit new page  2. Back  3. Forward  4. Show history  5. Back to main menu
Enter choice: 5

===== MAIN MENU =====
1. Stack
2. Queue
3. Circular Queue
4. Browser History
5. Exit
```

```
Enter your option: 5
Program ended
```

Program 4

Write a program to evaluate the Postfix Notation

Code:

```
expression = input("Enter postfix expression (separate with spaces): ")
tokens = expression.split()
stack = []
error = False

for t in tokens:
    if t.lstrip("-").isdigit():
        stack.append(int(t))
    elif t in "+-*/^":
        if len(stack) < 2:
            print("Invalid expression")
            error = True
            break
        b = stack.pop()
        a = stack.pop()
        if t == "+":
            r = a + b
        elif t == "-":
            r = a - b
        elif t == "*":
            r = a * b
        elif t == "/":
            if b == 0:
                print("Division by zero")
                error = True
                break
            r = a / b
        else:
            r = a ** b
        stack.append(r)
        print("After", a, t, b, "->", stack)
    else:
        print("Invalid token:", t)
        error = True
        break

if not error:
    if len(stack) == 1:
        print("Result =", stack[0])
    else:
        print("Invalid expression")
```

Output:

```
Enter postfix expression (separate with spaces): 2 3 1 * + 9 -
After 3 * 1 -> [2, 3]
After 2 + 3 -> [5]
After 5 - 9 -> [-4]
Result = -4
```

Program 5

Write a program to convert infix notation into Postfix Notation

Code:

```
def precedence(op):
    if op == "^":
        return 3
    if op == "*" or op == "/":
        return 2
    if op == "+" or op == "-":
        return 1
    return 0


infix = input("Enter infix expression: ").replace(" ", "")
stack = []
postfix = ""
valid = True

for ch in infix:
    if ch.isalnum():
        postfix += ch
    elif ch == "(":
        stack.append(ch)
    elif ch == ")":
        while len(stack) > 0 and stack[-1] != "(":
            postfix += stack.pop()
        if len(stack) == 0:
            valid = False
            break
        stack.pop()
    elif ch in "+-*/^":
        while len(stack) > 0 and stack[-1] != "(":
            top = stack[-1]
            if precedence(top) > precedence(ch) or (precedence(top) == precedence(ch) and ch != "^"):
                postfix += stack.pop()
            else:
                break
        stack.append(ch)
    else:
        valid = False
        break

while valid and len(stack) > 0:
    top = stack.pop()
    if top == "(":
        valid = False
    else:
        postfix += top

if valid:
    print("Postfix expression:", postfix)
else:
    print("Invalid expression")
```

Output:

```
Enter infix expression: a+b*(c^d-e)^(f+g*h)-i
Postfix expression: abcd^e-fgh*+^*+i-
```

Program 6

Write a program to perform the operations of the linked list: Insertion, Deletion, Display

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


head = None

while True:
    print("\n1. Insert at beginning")
    print("2. Insert at end")
    print("3. Insert at position")
    print("4. Delete from beginning")
    print("5. Delete from end")
    print("6. Delete by value")
    print("7. Display")
    print("8. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        new = Node(int(input("Enter value: ")))
        new.next = head
        head = new
    elif choice == 2:
        new = Node(int(input("Enter value: ")))
        if head is None:
            head = new
        else:
            temp = head
            while temp.next is not None:
                temp = temp.next
            temp.next = new
    elif choice == 3:
        value = int(input("Enter value: "))
        pos = int(input("Enter position (starting from 1): "))
        new = Node(value)
        if pos == 1:
            new.next = head
            head = new
        else:
            temp = head
            count = 1
            while temp is not None and count < pos - 1:
                temp = temp.next
                count += 1
            if temp is None:
                print("Invalid position")
            else:
                new.next = temp.next
                temp.next = new
    elif choice == 4:
        if head is None:
            print("List is empty")
        else:
            print("Deleted:", head.data)
            head = head.next
    elif choice == 5:
        if head is None:
            print("List is empty")
        elif head.next is None:
            print("Deleted:", head.data)
            head = None
        else:
            temp = head
            while temp.next.next is not None:
                temp = temp.next
            print("Deleted:", temp.next.data)
```

```
            temp.next = None
    elif choice == 6:
        value = int(input("Enter value to delete: "))
        if head is None:
            print("List is empty")
        elif head.data == value:
            head = head.next
            print("Deleted")
        else:
            temp = head
            while temp.next is not None and temp.next.data != value:
                temp = temp.next
            if temp.next is None:
                print("Value not found")
            else:
                temp.next = temp.next.next
                print("Deleted")
    elif choice == 7:
        if head is None:
            print("List is empty")
        else:
            temp = head
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")
    elif choice == 8:
        print("Program ended")
        break
    else:
        print("Invalid choice")
```

Output:

```
1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 1
Enter value: 10

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 2
Enter value: 20

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 2
Enter value: 30

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
```

```
8. Exit
Enter your choice: 1
Enter value: 5

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 3
Enter value: 25
Enter position (starting from 1): 3

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 7
5 -> 10 -> 25 -> 20 -> 30 -> None

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 100
Invalid choice

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 7
5 -> 10 -> 25 -> 20 -> 30 -> None

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 4
Deleted: 5

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 7
10 -> 25 -> 20 -> 30 -> None

1. Insert at beginning
2. Insert at end
```

```
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 5
Deleted: 30

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 7
10 -> 25 -> 20 -> None

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 6
Enter value to delete: 20
Deleted

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 6
Enter value to delete: 99
Value not found

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 7
10 -> 25 -> None

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display
8. Exit
Enter your choice: 8
Program ended
```

Program 7

Write a program to perform the operations of the Circular linked list: Insertion, Deletion, Display

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


head = None

while True:
    print("\n1. Insert at beginning")
    print("2. Insert at end")
    print("3. Delete from beginning")
    print("4. Delete from end")
    print("5. Delete by value")
    print("6. Display")
    print("7. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        new = Node(int(input("Enter value: ")))
        if head is None:
            head = new
            new.next = head
        else:
            last = head
            while last.next != head:
                last = last.next
            new.next = head
            last.next = new
            head = new
    elif choice == 2:
        new = Node(int(input("Enter value: ")))
        if head is None:
            head = new
            new.next = head
        else:
            last = head
            while last.next != head:
                last = last.next
            last.next = new
            new.next = head
    elif choice == 3:
        if head is None:
            print("List is empty")
        elif head.next == head:
            print("Deleted:", head.data)
            head = None
        else:
            last = head
            while last.next != head:
                last = last.next
            print("Deleted:", head.data)
            head = head.next
            last.next = head
    elif choice == 4:
        if head is None:
            print("List is empty")
        elif head.next == head:
            print("Deleted:", head.data)
            head = None
        else:
            prev = head
            while prev.next.next != head:
                prev = prev.next
            print("Deleted:", prev.next.data)
            prev.next = head
    elif choice == 5:
```

```
        value = int(input("Enter value to delete: "))
        if head is None:
            print("List is empty")
        elif head.data == value:
            if head.next == head:
                head = None
            else:
                last = head
                while last.next != head:
                    last = last.next
                head = head.next
                last.next = head
            print("Deleted")
        else:
            prev = head
            while prev.next != head and prev.next.data != value:
                prev = prev.next
            if prev.next == head:
                print("Value not found")
            else:
                prev.next = prev.next.next
                print("Deleted")
    elif choice == 6:
        if head is None:
            print("List is empty")
        else:
            temp = head
            while True:
                print(temp.data, end=" -> ")
                temp = temp.next
                if temp == head:
                    break
            print("(back to head)")
    elif choice == 7:
        print("Program ended")
        break
    else:
        print("Invalid choice")
```

Output:

```
1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 1
Enter value: 10

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 2
Enter value: 20

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 2
Enter value: 30

1. Insert at beginning
2. Insert at end
3. Delete from beginning
```

```
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 1
Enter value: 5

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 6
5 -> 10 -> 20 -> 30 -> (back to head)

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 3
Deleted: 5

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 6
10 -> 20 -> 30 -> (back to head)

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 4
Deleted: 30

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 6
10 -> 20 -> (back to head)

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 5
Enter value to delete: 20
Deleted

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
```

```
6. Display
7. Exit
Enter your choice: 6
10 -> (back to head)

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 5
Enter value to delete: 99
Value not found

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 6
10 -> (back to head)

1. Insert at beginning
2. Insert at end
3. Delete from beginning
4. Delete from end
5. Delete by value
6. Display
7. Exit
Enter your choice: 7
Program ended
```

Program 8

Write a program to perform all operations for Stack using Linked List, Queue using Linked List and Circular Queue using Linked List

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


def stack_menu():
    top = None
    while True:
        print("\n--- Stack using Linked List ---")
        print("1. Push  2. Pop  3. Peek  4. Display  5. Back")
        ch = int(input("Enter choice: "))
        if ch == 1:
            new = Node(int(input("Enter value: ")))
            new.next = top
            top = new
        elif ch == 2:
            if top is None:
                print("Stack Underflow")
            else:
                print("Popped:", top.data)
                top = top.next
        elif ch == 3:
            if top is None:
                print("Stack is empty")
            else:
                print("Top element:", top.data)
        elif ch == 4:
            if top is None:
                print("Stack is empty")
            else:
                temp = top
                print("Stack (top to bottom):", end=" ")
                while temp is not None:
                    print(temp.data, end=" ")
                    temp = temp.next
                print()
        elif ch == 5:
            break
        else:
            print("Invalid choice")


def queue_menu():
    front = None
    rear = None
    while True:
        print("\n--- Queue using Linked List ---")
        print("1. Enqueue  2. Dequeue  3. Display  4. Back")
        ch = int(input("Enter choice: "))
        if ch == 1:
            new = Node(int(input("Enter value: ")))
            if rear is None:
                front = new
                rear = new
            else:
                rear.next = new
                rear = new
        elif ch == 2:
            if front is None:
                print("Queue is empty")
            else:
                print("Dequeued:", front.data)
                front = front.next
                if front is None:
                    rear = None
```

```
        elif ch == 3:
            if front is None:
                print("Queue is empty")
            else:
                temp = front
                print("Queue:", end=" ")
                while temp is not None:
                    print(temp.data, end=" ")
                    temp = temp.next
                print()
        elif ch == 4:
            break
        else:
            print("Invalid choice")


def circular_menu():
    front = None
    rear = None
    while True:
        print("\n--- Circular Queue using Linked List ---")
        print("1. Enqueue  2. Dequeue  3. Display  4. Back")
        ch = int(input("Enter choice: "))
        if ch == 1:
            new = Node(int(input("Enter value: ")))
            if front is None:
                front = new
                rear = new
                rear.next = front
            else:
                rear.next = new
                rear = new
                rear.next = front
        elif ch == 2:
            if front is None:
                print("Circular queue is empty")
            else:
                print("Dequeued:", front.data)
                if front == rear:
                    front = None
                    rear = None
                else:
                    front = front.next
                    rear.next = front
        elif ch == 3:
            if front is None:
                print("Circular queue is empty")
            else:
                temp = front
                print("Circular queue:", end=" ")
                while True:
                    print(temp.data, end=" ")
                    temp = temp.next
                    if temp == front:
                        break
                print()
        elif ch == 4:
            break
        else:
            print("Invalid choice")


while True:
    print("\n===== MAIN MENU =====")
    print("1. Stack")
    print("2. Queue")
    print("3. Circular Queue")
    print("4. Exit")
    option = int(input("Enter your option: "))
    if option == 1:
        stack_menu()
    elif option == 2:
        queue_menu()
```

```
    elif option == 3:
        circular_menu()
    elif option == 4:
        print("Program ended")
        break
    else:
        print("Invalid option")
```

Output:

```
===== MAIN MENU =====
1. Stack
2. Queue
3. Circular Queue
4. Exit
Enter your option: 1

--- Stack using Linked List ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 1
Enter value: 10

--- Stack using Linked List ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 1
Enter value: 20

--- Stack using Linked List ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 1
Enter value: 30

--- Stack using Linked List ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 4
Stack (top to bottom): 30 20 10

--- Stack using Linked List ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 3
Top element: 30

--- Stack using Linked List ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 2
Popped: 30

--- Stack using Linked List ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 4
Stack (top to bottom): 20 10

--- Stack using Linked List ---
1. Push  2. Pop  3. Peek  4. Display  5. Back
Enter choice: 5

===== MAIN MENU =====
1. Stack
2. Queue
3. Circular Queue
4. Exit
Enter your option: 2

--- Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 11

--- Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 22

--- Queue using Linked List ---
```

```
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 33

--- Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 3
Queue: 11 22 33

--- Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 2
Dequeued: 11

--- Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 3
Queue: 22 33

--- Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 4

===== MAIN MENU =====
1. Stack
2. Queue
3. Circular Queue
4. Exit
Enter your option: 3

--- Circular Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 1

--- Circular Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 2

--- Circular Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 3

--- Circular Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 3
Circular queue: 1 2 3

--- Circular Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 2
Dequeued: 1

--- Circular Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 3
Circular queue: 2 3

--- Circular Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 1
Enter value: 4

--- Circular Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 3
Circular queue: 2 3 4

--- Circular Queue using Linked List ---
1. Enqueue  2. Dequeue  3. Display  4. Back
Enter choice: 4
```

```
===== MAIN MENU =====
1. Stack
2. Queue
3. Circular Queue
4. Exit
Enter your option: 4
Program ended
```

Program 9

Write a program to perform concatenation of two linked lists and implement Polynomial Representation and Addition using Linked Lists

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Term:
    def __init__(self, coef, exp):
        self.coef = coef
        self.exp = exp
        self.next = None


def build_list(count):
    head = None
    for i in range(count):
        new = Node(int(input("Enter element " + str(i + 1) + ": ")))
        if head is None:
            head = new
        else:
            temp = head
            while temp.next is not None:
                temp = temp.next
            temp.next = new
    return head


def show_list(head):
    if head is None:
        print("List is empty")
        return
    temp = head
    while temp is not None:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")


def read_poly(name):
    head = None
    tail = None
    n = int(input("Enter number of terms in " + name + ": "))
    print("Enter terms in decreasing order of exponent")
    for i in range(n):
        c = int(input("Coefficient of term " + str(i + 1) + ": "))
        e = int(input("Exponent of term " + str(i + 1) + ": "))
        new = Term(c, e)
        if head is None:
            head = new
            tail = new
        else:
            tail.next = new
            tail = new
    return head


def show_poly(head):
    if head is None:
        print("0")
        return
    temp = head
    text = ""
    while temp is not None:
        if temp.coef >= 0 and text != "":
            text += " + "
        elif temp.coef < 0:
```

```
            text += " - " if text != "" else "-"
        value = abs(temp.coef)
        if temp.exp == 0:
            text += str(value)
        elif temp.exp == 1:
            text += str(value) + "x"
        else:
            text += str(value) + "x^" + str(temp.exp)
        temp = temp.next
    print(text)


def add_poly(p1, p2):
    result = None
    tail = None
    while p1 is not None or p2 is not None:
        if p2 is None or (p1 is not None and p1.exp > p2.exp):
            new = Term(p1.coef, p1.exp)
            p1 = p1.next
        elif p1 is None or p2.exp > p1.exp:
            new = Term(p2.coef, p2.exp)
            p2 = p2.next
        else:
            new = Term(p1.coef + p2.coef, p1.exp)
            p1 = p1.next
            p2 = p2.next
        if new.coef == 0:
            continue
        if result is None:
            result = new
            tail = new
        else:
            tail.next = new
            tail = new
    return result


print("===== Concatenation of two linked lists =====")
n1 = int(input("Enter number of elements in first list: "))
list1 = build_list(n1)
n2 = int(input("Enter number of elements in second list: "))
list2 = build_list(n2)

print("First list:")
show_list(list1)
print("Second list:")
show_list(list2)

if list1 is None:
    joined = list2
else:
    temp = list1
    while temp.next is not None:
        temp = temp.next
    temp.next = list2
    joined = list1

print("Concatenated list:")
show_list(joined)

print("\n===== Polynomial addition =====")
poly1 = read_poly("first polynomial")
poly2 = read_poly("second polynomial")

print("First polynomial :", end=" ")
show_poly(poly1)
print("Second polynomial:", end=" ")
show_poly(poly2)
print("Sum              :", end=" ")
show_poly(add_poly(poly1, poly2))
```

Output:

```
===== Concatenation of two linked lists =====
```

```
Enter number of elements in first list: 3
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter number of elements in second list: 2
Enter element 1: 40
Enter element 2: 50
First list:
10 -> 20 -> 30 -> None
Second list:
40 -> 50 -> None
Concatenated list:
10 -> 20 -> 30 -> 40 -> 50 -> None

===== Polynomial addition =====
Enter number of terms in first polynomial: 3
Enter terms in decreasing order of exponent
Coefficient of term 1: 2
Exponent of term 1: 6
Coefficient of term 2: 5
Exponent of term 2: 3
Coefficient of term 3: 4
Exponent of term 3: 1
Enter number of terms in second polynomial: 4
Enter terms in decreasing order of exponent
Coefficient of term 1: 3
Exponent of term 1: 4
Coefficient of term 2: -5
Exponent of term 2: 3
Coefficient of term 3: 1
Exponent of term 3: 2
Coefficient of term 4: 2
Exponent of term 4: 0
First polynomial : 2x^6 + 5x^3 + 4x
Second polynomial: 3x^4 - 5x^3 + 1x^2 + 2
Sum              : 2x^6 + 3x^4 + 1x^2 + 4x + 2
```

Program 10

Write a program to perform the operations of the Double linked list: Insertion, Deletion, Display

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


head = None

while True:
    print("\n1. Insert at beginning")
    print("2. Insert at end")
    print("3. Insert at position")
    print("4. Delete from beginning")
    print("5. Delete from end")
    print("6. Delete by value")
    print("7. Display forward")
    print("8. Display backward")
    print("9. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        new = Node(int(input("Enter value: ")))
        new.next = head
        if head is not None:
            head.prev = new
        head = new
    elif choice == 2:
        new = Node(int(input("Enter value: ")))
        if head is None:
            head = new
        else:
            temp = head
            while temp.next is not None:
                temp = temp.next
            temp.next = new
            new.prev = temp
    elif choice == 3:
        value = int(input("Enter value: "))
        pos = int(input("Enter position (starting from 1): "))
        new = Node(value)
        if pos == 1:
            new.next = head
            if head is not None:
                head.prev = new
            head = new
        else:
            temp = head
            count = 1
            while temp is not None and count < pos - 1:
                temp = temp.next
                count += 1
            if temp is None:
                print("Invalid position")
            else:
                new.next = temp.next
                new.prev = temp
                if temp.next is not None:
                    temp.next.prev = new
                temp.next = new
    elif choice == 4:
        if head is None:
            print("List is empty")
        else:
            print("Deleted:", head.data)
            head = head.next
            if head is not None:
```

```
            head.prev = None
    elif choice == 5:
        if head is None:
            print("List is empty")
        elif head.next is None:
            print("Deleted:", head.data)
            head = None
        else:
            temp = head
            while temp.next is not None:
                temp = temp.next
            print("Deleted:", temp.data)
            temp.prev.next = None
    elif choice == 6:
        value = int(input("Enter value to delete: "))
        temp = head
        while temp is not None and temp.data != value:
            temp = temp.next
        if temp is None:
            print("Value not found")
        else:
            if temp.prev is not None:
                temp.prev.next = temp.next
            else:
                head = temp.next
            if temp.next is not None:
                temp.next.prev = temp.prev
            print("Deleted")
    elif choice == 7:
        if head is None:
            print("List is empty")
        else:
            temp = head
            print("None", end=" <-> ")
            while temp is not None:
                print(temp.data, end=" <-> ")
                temp = temp.next
            print("None")
    elif choice == 8:
        if head is None:
            print("List is empty")
        else:
            temp = head
            while temp.next is not None:
                temp = temp.next
            print("None", end=" <-> ")
            while temp is not None:
                print(temp.data, end=" <-> ")
                temp = temp.prev
            print("None")
    elif choice == 9:
        print("Program ended")
        break
    else:
        print("Invalid choice")
```

Output:

```
1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 1
Enter value: 10

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
```

```
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 2
Enter value: 20

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 2
Enter value: 30

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 3
Enter value: 25
Enter position (starting from 1): 3

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 7
None <-> 10 <-> 20 <-> 25 <-> 30 <-> None

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 100
Invalid choice

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 7
None <-> 10 <-> 20 <-> 25 <-> 30 <-> None

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
```

```
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 8
None <-> 30 <-> 25 <-> 20 <-> 10 <-> None

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 4
Deleted: 10

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 5
Deleted: 30

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 6
Enter value to delete: 20
Deleted

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 7
None <-> 25 <-> None

1. Insert at beginning
2. Insert at end
3. Insert at position
4. Delete from beginning
5. Delete from end
6. Delete by value
7. Display forward
8. Display backward
9. Exit
Enter your choice: 9
Program ended
```